A Linux Kernel Congestion Control Module for the Assignment 2 Design

Assignment 3

Marko Kovacevic

CS 42200: Computer Networks
Purdue University · Fall 2026
Instructor: Vamsi Addanki
TAs: Youngsuk Kim, Xiao Luo, and Albert Vo

October 24, 2026

Abstract

The congestion-window rule from Assignment 2 was implemented as a Linux kernel module named `kovacevic` and measured against CUBIC, Reno, and BBR with the Assignment 2 client. The module doubles its window in startup, pauses when the smoothed RTT exceeds the path minimum by more than 20%, cuts once by 10% on a flat high-delay round, and halves the window on loss. Ten-second transfers ran twice per algorithm at ten public iperf3 destinations. The median goodput across run medians was 105.0 Mbit/s for kovacevic, 104.0 for Reno, 101.3 for BBR, and 100.4 for CUBIC. kovacevic recorded 3 retransmission intervals against Reno's 32, CUBIC's 160, and BBR's 4.

1. Introduction

This assignment implements the Assignment 2 window-update rule as a Linux congestion control named `kovacevic`. The kernel calls the module on each ACK and on loss, and the module reads the smoothed RTT from the socket. The same Python iperf3 client from Assignment 2 measures the module alongside CUBIC, Reno, and BBR.

2. Implementation

2.1. How the module updates the congestion window

Linux congestion controls plug into `struct tcp_congestion_ops` and receive callbacks from the ACK path. The module registers the name `kovacevic` and implements two callbacks, `kov_cong_avoid` for window growth and `kov_ssthresh` for loss. The kernel invokes `kov_cong_avoid` for every ACK that delivers new data, with the newly acknowledged count given in segments, and `kov_ssthresh` when a loss recovery starts.

The Assignment 2 rule decided once per RTT. The module has no per-RTT hook, so it decides once per round. A round ends when the segments acknowledged since the previous decision reach the current window, which is one RTT of ACKs for a window-limited sender. Each completed round produces one RTT and one rate sample.

- **RTT.** `srtt_us` holds the kernel's smoothed RTT in 1/8 microseconds. The module keeps its own minimum and a 3:1 smoothed value in the same 75% / 25% mix as the Assignment 2 pseudocode.
- **Throughput.** The round's rate is `acked segments × MSS / round duration`, smoothed 3:1. A round is flat when the smoothed rate grows by less than 5% over the previous one.
- **Loss.** `kov_ssthresh` returns half the current window, at least two segments, and moves the module into avoidance. The kernel's own loss detection triggers this branch, so the module does not count retransmissions itself.

The window decisions follow the Assignment 2 pseudocode. In startup the window doubles each round. When the queue delay, smoothed RTT minus the minimum, exceeds the larger of 2 ms and 20% of the minimum RTT, growth pauses. Two consecutive flat rounds end startup. If the delay is high at that moment the window is cut by 10% once, and the cut stays used until delay falls below the threshold. In avoidance the window grows by one segment per round while delay stays low, and the same one-time 10% cut applies to flat high-delay rounds. Growth is capped by the kernel's `snd_cwnd_clamp`, so the receiver window still bounds the sender.

```
per ACK:
    if first ACK: mode = startup; ssthresh = infinite; start round clock
    if not cwnd-limited: return
    round_acked += acked
    if round_acked < cwnd: return
    dt = now - round_start; round_acked = 0
    rtt = srtt_us / 8
    if rtt == 0 or dt == 0: flat_rounds = 0; return
    if min_rtt unset: min_rtt = smoothed_rtt = rtt
    else: min_rtt = min(min_rtt, rtt); smoothed_rtt = (3·smoothed_rtt + rtt)/4
    rate = acked_segments · mss · 8 / dt
    smoothed_rate = (3·smoothed_rate + rate)/4, or rate if unset
    if prev_rate > 0 and smoothed_rate < 1.05 · prev_rate:
         flat_rounds = min(2, flat_rounds + 1)
    else: flat_rounds = 0
    prev_rate = smoothed_rate
    queue_delay = smoothed_rtt - min_rtt
    queue_limit = max(2 ms, 20% of min_rtt)
    if mode == startup and flat_rounds == 2:
         mode = avoidance
         if queue_delay > queue_limit: cwnd = max(2, 0.90·cwnd); delay_cut_used
    elif mode == startup and queue_delay > queue_limit: hold
    elif mode == startup: cwnd = 2·cwnd
    elif queue_delay > queue_limit:
         if flat_rounds == 2 and not delay_cut_used:
              cwnd = max(2, 0.90·cwnd); delay_cut_used
         else: hold
    else: delay_cut_used = false; cwnd = cwnd + 1

on loss entry: mode = avoidance; flat_rounds = 0; delay_cut_used = true
               return max(2, cwnd / 2)
```

2.2. Initial state and parameters

A socket starts in startup with `flat_rounds = 0`, `delay_cut_used = false`, and the minimum RTT and smoothed values unset. The first valid round fills them in. The window starts at the kernel's ten-segment initial value. The slow-start threshold is set to the kernel's infinite value so the kernel's own slow-start logic does not add growth on top.

Parameter	Value	Meaning
Minimum window	2 segments	lower bound after any cut
Startup growth	double per round	until two flat rounds
Avoidance growth	+1 segment per round	while queue delay stays low
Loss response	halve the window	`kov_ssthresh` callback, at least 2 segments
Delay pause	queue delay > max(2 ms, 20% of min RTT)	hold the window
Delay cut	10%, once until delay falls	on flat high-delay rounds
Flat round	smoothed rate < 1.05 × previous	counts toward startup exit
Smoothing	75% old / 25% new	RTT and rate estimates

2.3. Changes from the Assignment 2 pseudocode

The rule itself is unchanged. The differences are in how the kernel delivers its inputs.

1. **Once per RTT becomes once per round.** The kernel calls the module per ACK, so a round of acknowledged segments reaching the current window substitutes for one RTT of decisions.
2. **The initial window is the kernel's, not two segments.** The kernel sets its ten-segment initial window during connection setup, after the module's setup already ran. The module's logic starts from whatever window exists at the first ACK.
3. **The rate sample is per round.** The Assignment 2 client sampled acknowledged bytes every half second. The module computes each round's rate as acknowledged segments times MSS divided by the round duration, the same quantity at a coarser resolution.
4. **RTT comes from the kernel's estimate.** `srtt_us` is the kernel's smoothed RTT in 1/8 microsecond units. The module divides by eight and re-smooths 3:1 per round. The pseudocode's invalid-RTT branch becomes `rtt == 0 or dt == 0`, which holds the window.
5. **Loss enters through `kov_ssthresh`.** The pseudocode compared retransmission counters each round. The kernel instead calls `kov_ssthresh` when its loss recovery starts, so the halving and the state changes happen there.
6. **Undo uses the kernel default.** A spurious loss recovery is undone with the kernel's standard `tcp_reno_undo_cwnd`, which restores at most twice the slow-start threshold. The pseudocode did not specify an undo.
7. **The window-limited case is a gate, not a state.** Where the Assignment 2 report noted that a receiver-limited sender must not grow, the module returns without a decision whenever `tcp_is_cwnd_limited` is false.

2.4. Build and load

The module builds out of tree against the running kernel's headers. It was built and tested on Linux kernel 6.17.0-35-generic. It needs gcc, make, and the `linux-headers` package for the running kernel. No special kernel features are required beyond module loading, which works on this machine because Secure Boot is disabled.

```
make
sudo insmod tcp_kovacevic.ko
sysctl net.ipv4.tcp_available_congestion_control
sudo rmmod tcp_kovacevic
```

After loading, the `sysctl` output ends with `kovacevic`. The measurement client sets `TCP_CONGESTION` to `kovacevic` on each data socket before connecting and reads the selection back, and every saved sample records `"algorithm": "kovacevic"`. The grading environment does not load kernel modules, so the module was built and run on this machine only. `demo.py` runs a single ten-second transfer and prints the window, RTT, retransmissions, and goodput once per half second, showing the startup doubling and the delay pause.

3. Measurements

3.1. Setup

The Assignment 2 client opens the iperf3 control connection, exchanges parameters, and sends data on a socket whose congestion control is selected before connecting. Nine of the ten Assignment 2 destinations completed all four algorithms. The Nanjing server stalled on the first kovacevic transfer, its data socket made no progress, and the client's stall guard replaced it with `speedtest.xmission.com`, the next candidate in the recorded order. Every transfer lasted 10 seconds with samples every 0.5 seconds, and each algorithm ran 2 repetitions. The four test orders rotated per destination and repetition.

kovacevic–cubic–reno–bbr, cubic–reno–bbr–kovacevic, reno–bbr–kovacevic–cubic, bbr–kovacevic–cubic–reno

Each sample carries the congestion window in segments, the smoothed RTT, the retransmission count since the previous sample as the loss signal, and acknowledged goodput. All four come from `TCP_INFO` on the data socket

through `tcp_stats.py`. The host ran kernel 6.17.0-35-generic with CUBIC, Reno, and BBR available in addition to the loaded module, without a VPN.

3.2. AI acknowledgment

I used Codex while porting the Assignment 2 rule into the kernel and drafting this report. I built and loaded the module on this machine, worked through the ACK-path callbacks against the kernel headers, and checked the numbers in this report against the saved samples.

4. Results

4.1. Goodput across destinations

The two figures show acknowledged goodput over time for each destination. Solid lines are the first run and dashed lines the second. The table gives the minimum, median, mean, and 95th-percentile interval goodput for every destination and algorithm, pooling the half-second samples of both runs. The figures and table are generated by `plot_throughput` and `summarize`.

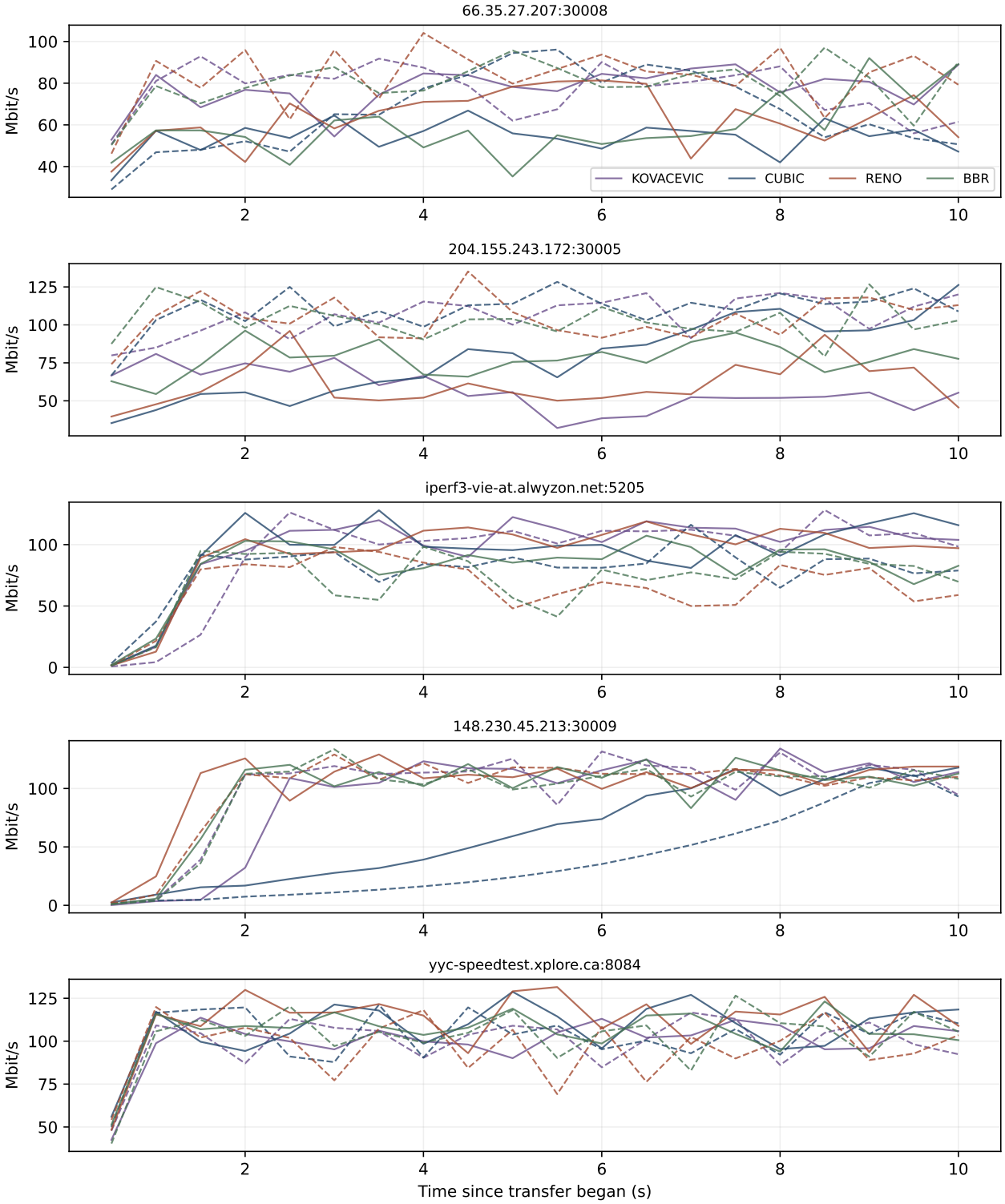


Figure 1: Acknowledged goodput over time for destinations 1–5, all four algorithms.

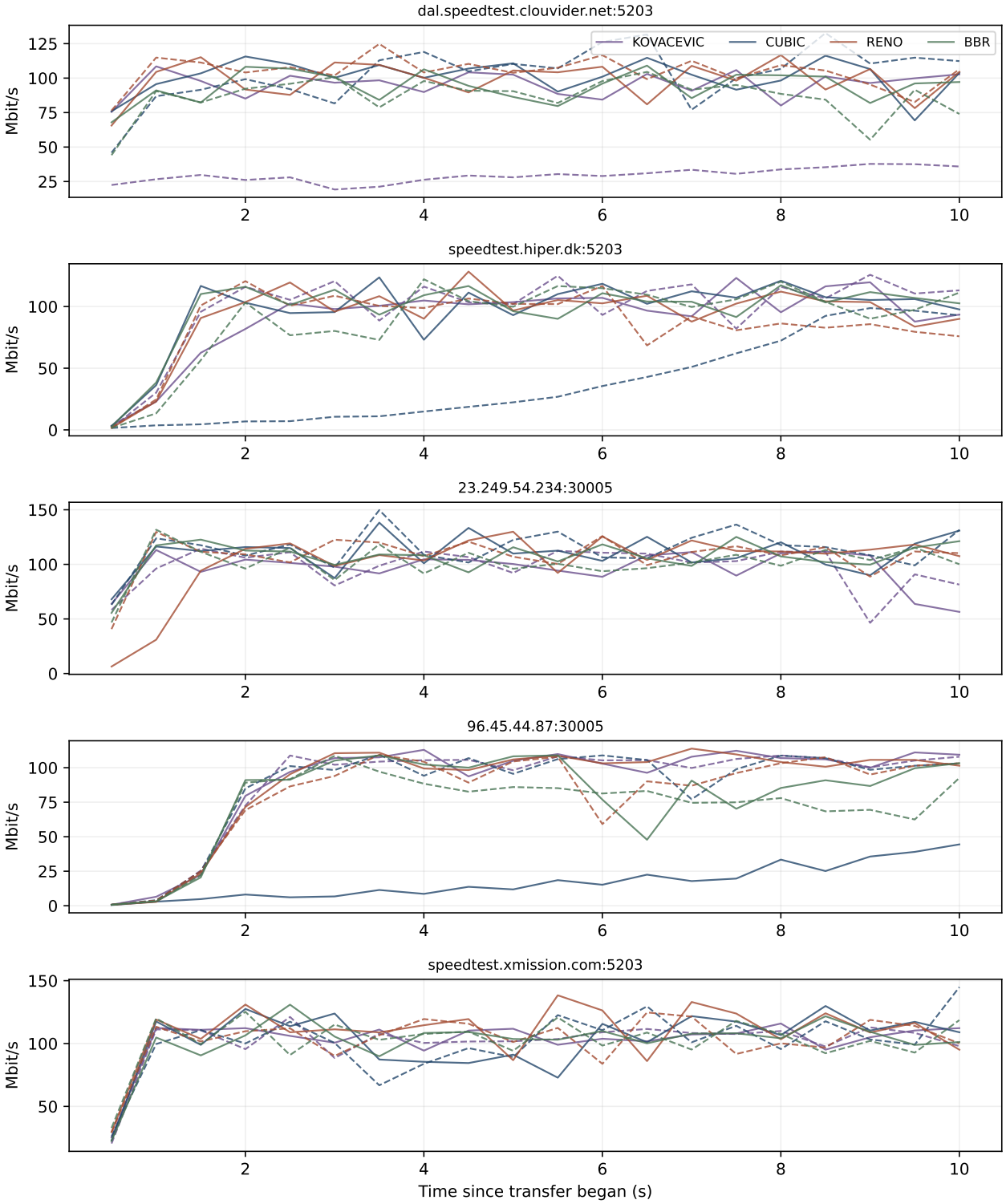


Figure 2: Acknowledged goodput over time for destinations 6–10, all four algorithms.

Acknowledged goodput (Mbit/s)

Destination	Port	TCP	Min	Median	Mean	P95
66.35.27.207	30008	kovacevic	50.7	80.3	77.1	90.2
66.35.27.207	30008	cubic	29.0	57.1	60.2	89.2
66.35.27.207	30008	reno	37.6	76.0	73.3	96.0
66.35.27.207	30008	bbr	35.2	73.2	69.5	92.2
204.155.243.172	30005	kovacevic	32.1	80.4	81.7	120.1
204.155.243.172	30005	cubic	35.3	102.7	94.0	125.1
204.155.243.172	30005	reno	39.6	91.3	82.6	118.3
204.155.243.172	30005	bbr	54.4	90.4	90.3	115.5
iperf3-vie-at.alwyzon.net	5205	kovacevic	0.7	106.3	95.2	122.6
iperf3-vie-at.alwyzon.net	5205	cubic	1.5	90.0	86.7	125.6
iperf3-vie-at.alwyzon.net	5205	reno	1.6	87.3	79.8	113.0
iperf3-vie-at.alwyzon.net	5205	bbr	1.3	84.3	76.2	102.6
148.230.45.213	30009	kovacevic	0.2	112.6	95.6	130.9
148.230.45.213	30009	cubic	1.6	41.1	51.9	116.7
148.230.45.213	30009	reno	1.6	112.0	100.9	126.0
148.230.45.213	30009	bbr	0.9	109.6	96.6	125.1
yyc-speedtest.xplore.ca	8084	kovacevic	42.5	104.0	99.8	113.1
yyc-speedtest.xplore.ca	8084	cubic	50.0	109.5	105.5	121.6
yyc-speedtest.xplore.ca	8084	reno	48.3	107.6	104.2	129.1
yyc-speedtest.xplore.ca	8084	bbr	40.3	105.7	104.1	120.4
dal.speedtest.clouvider.net	5203	kovacevic	19.2	56.6	62.6	104.3
dal.speedtest.clouvider.net	5203	cubic	45.9	104.3	102.1	126.4
dal.speedtest.clouvider.net	5203	reno	65.6	104.5	101.9	116.6
dal.speedtest.clouvider.net	5203	bbr	44.1	91.6	90.2	106.9
speedtest.hiper.dk	5203	kovacevic	1.6	103.0	95.1	123.3
speedtest.hiper.dk	5203	cubic	1.5	92.6	67.8	118.6
speedtest.hiper.dk	5203	reno	1.6	99.7	89.8	119.6
speedtest.hiper.dk	5203	bbr	1.6	103.6	93.5	117.4
23.249.54.234	30005	kovacevic	46.4	102.3	96.8	112.4
23.249.54.234	30005	cubic	63.0	113.8	112.3	136.5
23.249.54.234	30005	reno	6.4	110.8	105.1	126.1
23.249.54.234	30005	bbr	46.8	106.4	104.4	122.8
96.45.44.87	30005	kovacevic	0.7	105.2	89.4	111.1
96.45.44.87	30005	cubic	0.5	34.5	52.0	108.7
96.45.44.87	30005	reno	0.7	101.1	85.3	110.5
96.45.44.87	30005	bbr	0.7	85.6	75.8	108.6
speedtest.xmission.com	5203	kovacevic	20.5	107.7	102.0	113.0
speedtest.xmission.com	5203	cubic	23.0	105.0	102.1	129.6
speedtest.xmission.com	5203	reno	29.4	110.6	106.3	131.1
speedtest.xmission.com	5203	bbr	22.5	105.2	102.2	121.9

Each row pools the half-second samples from two completed runs.

4.2. RTT and loss

The table gives the pooled median RTT per destination and algorithm. kovacevic's median RTT was below Reno's at eight of ten destinations and within 6 ms of it at the other two. Against CUBIC it was lower at six destinations and higher at the four long-RTT paths, by up to 26 ms, where its standing window kept a queue that CUBIC did not maintain.

Median RTT (ms)

Destination	kovacevic	CUBIC	Reno	BBR
66.35.27.207	53.1	58.5	66.5	45.8
204.155.243.172	40.0	43.9	55.3	34.6
iperf3-vie-at.alwyzon.net	162.3	136.1	157.1	134.3
148.230.45.213	150.0	126.6	146.8	132.6
yyc-speedtest.xplore.ca	55.9	64.5	74.4	57.3
dal.speedtest.clouvider.net	38.6	49.5	58.2	37.8
speedtest.hiper.dk	132.5	121.7	145.6	124.7
23.249.54.234	52.6	63.0	65.3	52.8
96.45.44.87	203.3	198.8	205.9	200.5
speedtest.xmission.com	76.9	78.6	102.6	67.5

Each value pools the half-second samples from two completed runs.

The loss counts are small everywhere. Across the complete destinations kovacevic recorded 3 retransmission intervals, one each at Toronto, the Dallas server, and the Miami server. BBR recorded 4, Reno 32, and CUBIC 160. All but one of CUBIC's intervals came from a single Copenhagen run, and its other nineteen runs recorded none, so these totals are single events rather than steady per-algorithm behavior. The heaviest loss was on the Nanjing path that failed the comparison. The one BBR run saved there recorded 946 retransmissions before the following kovacevic transfer stalled, which points to a broken path rather than an algorithm difference. The delay pause kept kovacevic's window below the loss point on every completed destination except those three single events.

4.3. Aggregate statistics

The aggregates summarize each algorithm over all ten destinations. The first column takes the median goodput within each run and then the median across the twenty runs, so no destination or run dominates. The RTT column pools all half-second samples.

Algorithm	Median goodput, run medians (Mbit/s)	Mean goodput, run means (Mbit/s)	Pooled median RTT (ms)	Retransmissions
kovacevic	105.0	89.5	70.4	3
CUBIC	100.4	83.5	76.1	160
Reno	104.0	92.9	98.0	32
BBR	101.3	90.3	65.9	4

Computed from the saved samples. Run medians and run means are the same per-run statistics used in the Assignment 2 report.

kovacevic's median goodput across run medians was 105.0 Mbit/s against Reno's 104.0, BBR's 101.3, and CUBIC's 100.4. That is a tie with Reno for practical purposes, not a win. Reno's mean goodput was the highest, 92.9 against kovacevic's 89.5, because Reno's larger windows sustained higher rates after startup. kovacevic's RTT sat between BBR's and CUBIC's, and its retransmission total was the lowest of the four, 3 against BBR's 4, Reno's 32, and CUBIC's 160.

4.4. The long-RTT path

The Sydney destination (`96.45.44.87`) has a minimum RTT of about 187 ms. At kovacevic's 105 Mbit/s that RTT gives a bandwidth-delay product of roughly 2.4 MB, or 1,700 segments at the 1,448-byte MSS. kovacevic doubled from its initial window and reached 5,120 segments in both runs, with median goodputs of 105.5 and 105.2 Mbit/s and no retransmissions. CUBIC reached only 14.4 Mbit/s in the first run, its window still growing from 32 to 840 segments at the end, and 100.0 Mbit/s in the second. Reno managed 103.6 and 94.5, BBR 82.0 and 91.0. The slow ramp that motivated the Assignment 2 design reappeared in one CUBIC run and not in either kovacevic run.

The 5,120-segment window is about three times the operating-point bandwidth-delay product, and the median RTT rose to about 203 ms from a minimum near 187. The path buffers absorbed the excess without loss. The delivery rate stopped growing at the server's rate limit, so the flat-rate check ended the startup and the module held its window instead of cutting. The 5,120 segments were not the receiver's limit. BBR reached 5,609 segments on the same destination, so the halt came from the module, not from the advertised window. On this path the overshoot cost about 16 ms of queueing and no goodput.

4.5. Startup, pause, and cut in the traces

Toronto is the Assignment 2 representative destination and is used again here. All four algorithms are plotted there with common axis scales, generated by `plot_algorithm_time_series` and `plot_algorithm_scatter`.

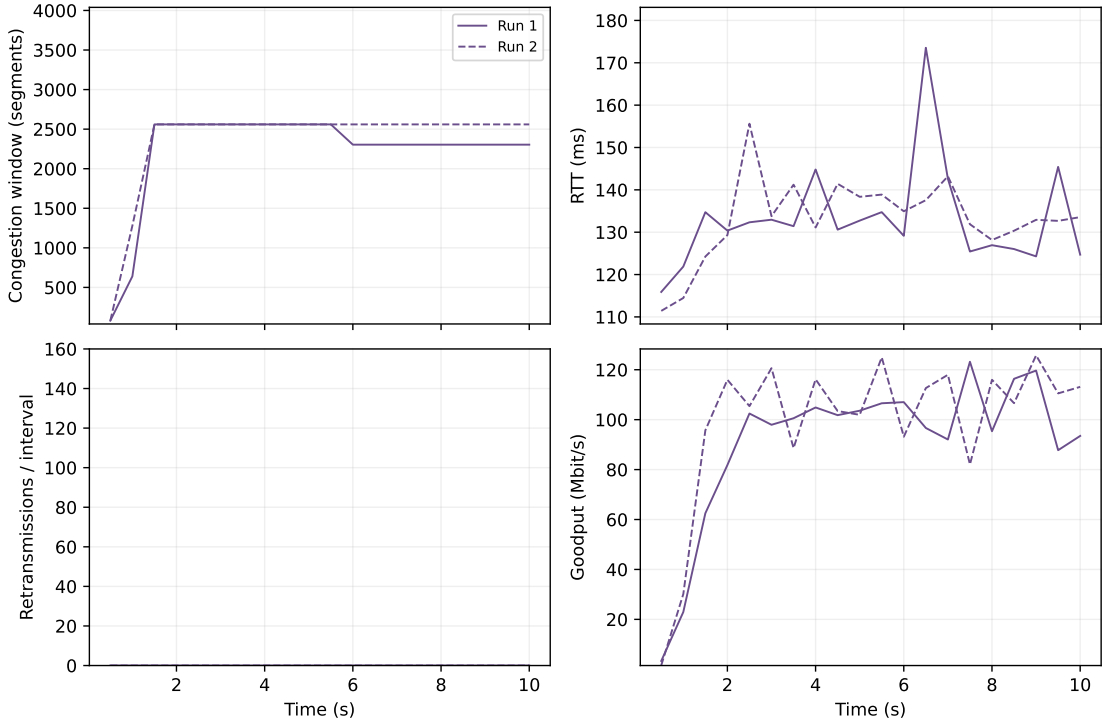


Figure 3: kovacevic data-socket congestion window, RTT, retransmissions per sample, and goodput over time at Toronto.

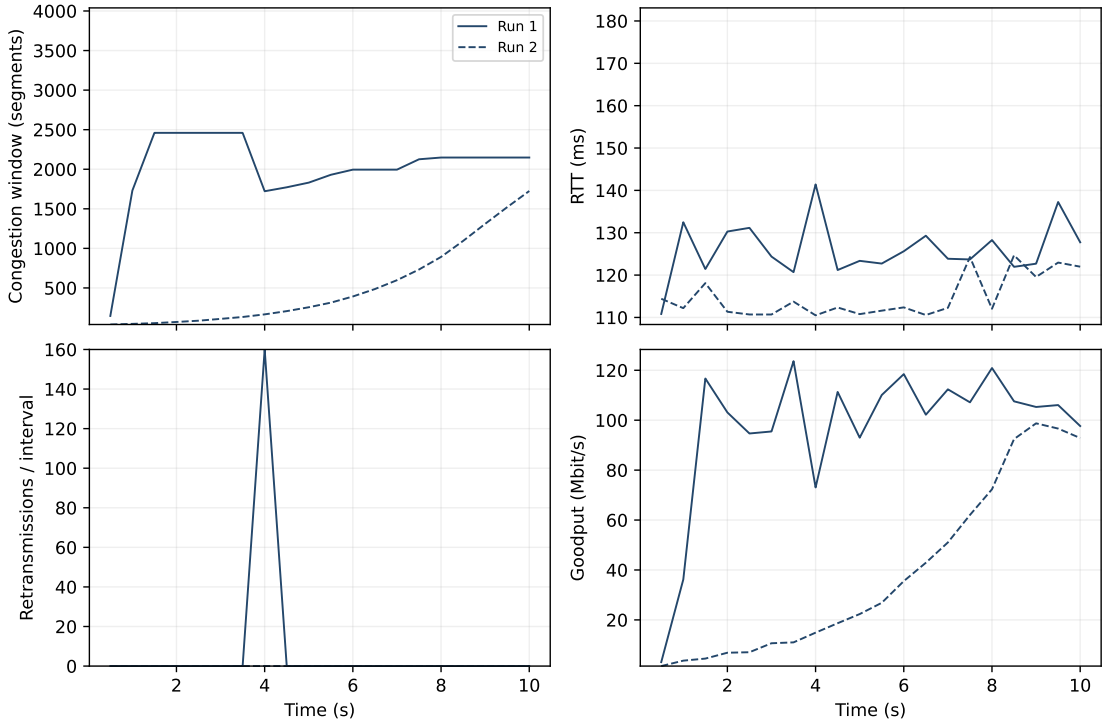


Figure 4: CUBIC congestion window, RTT, retransmissions per sample, and goodput over time at Toronto.

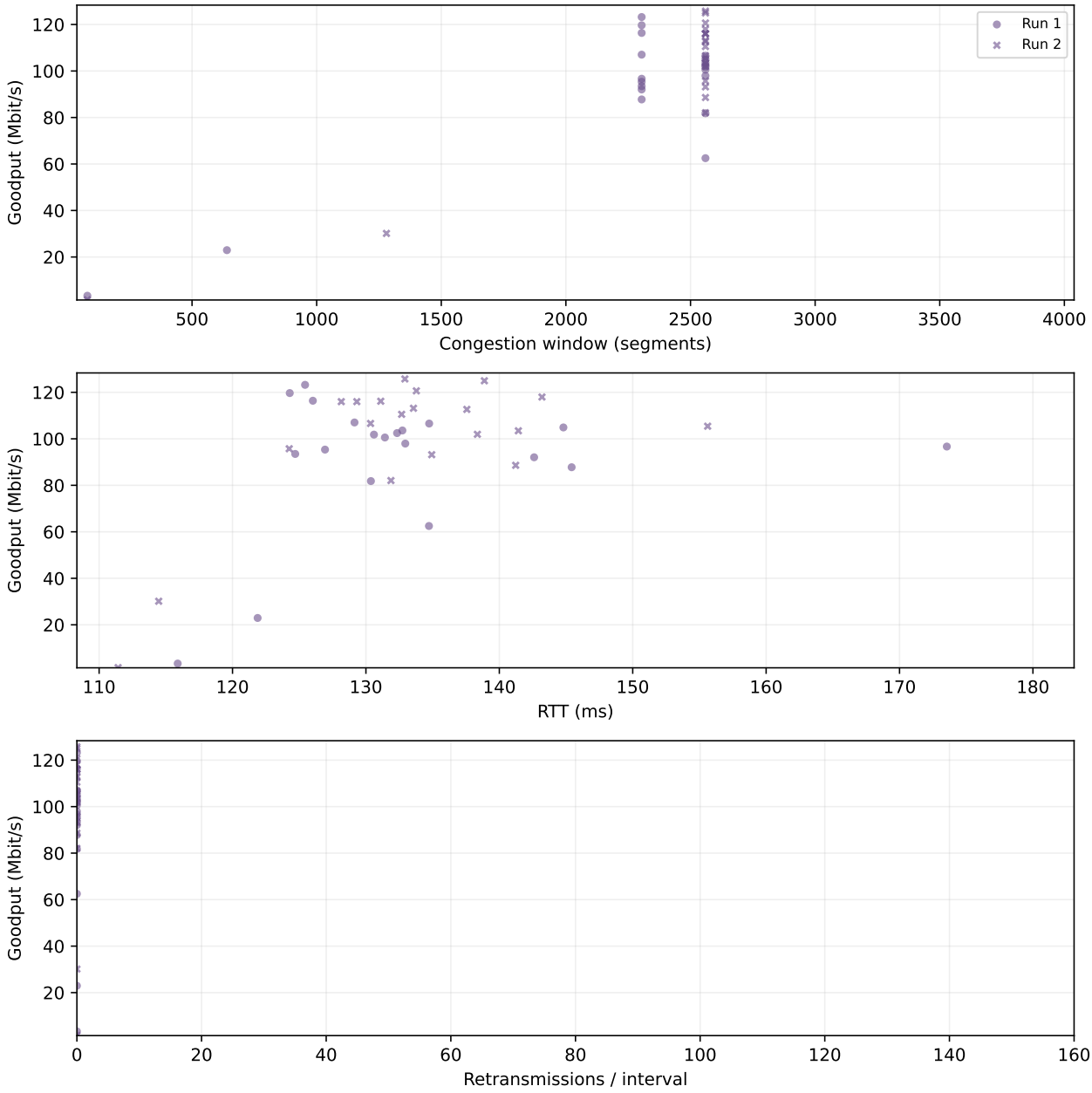


Figure 5: kovacevic congestion window, RTT, and retransmission signal versus acknowledged goodput at Toronto.

Both kovacevic runs show their held window in the first sample. The first run reaches 640 segments and the startup-exit cut reduces it to 576. The second run follows the same path from 320 to 288. The first run reached 110.2 Mbit/s median goodput with a 39.4 ms median RTT and no retransmissions. The second reached 55.4 Mbit/s after one retransmission, with 40.5 ms. CUBIC grew to windows of 1,600 to 2,000 segments and reached 113.3 and 82.7 Mbit/s. Reno kept windows of 1,400 to 2,700 segments, ran at 55.5 and 105.2 Mbit/s, and ran with 63.4 and 52.4 ms median RTTs. BBR reached 102.2 and 77.1 Mbit/s with the lowest RTTs of the four, 32.3 and 37.4 ms. In the first run kovacevic was close to CUBIC and BBR. In the second run it was behind all three. Its RTT stayed below Reno's throughout.

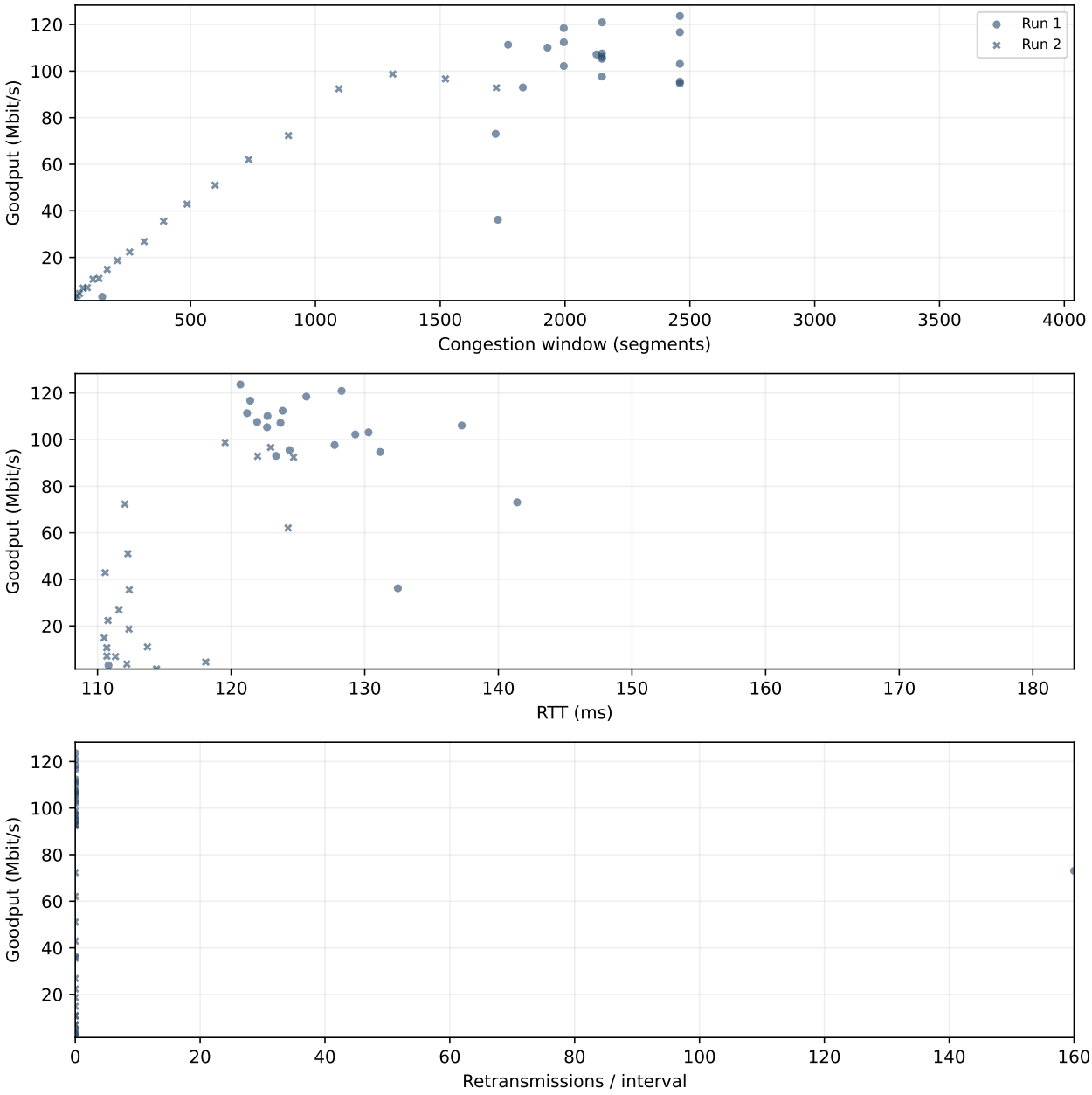


Figure 6: CUBIC congestion window, RTT, and retransmission signal versus acknowledged goodput at Toronto.

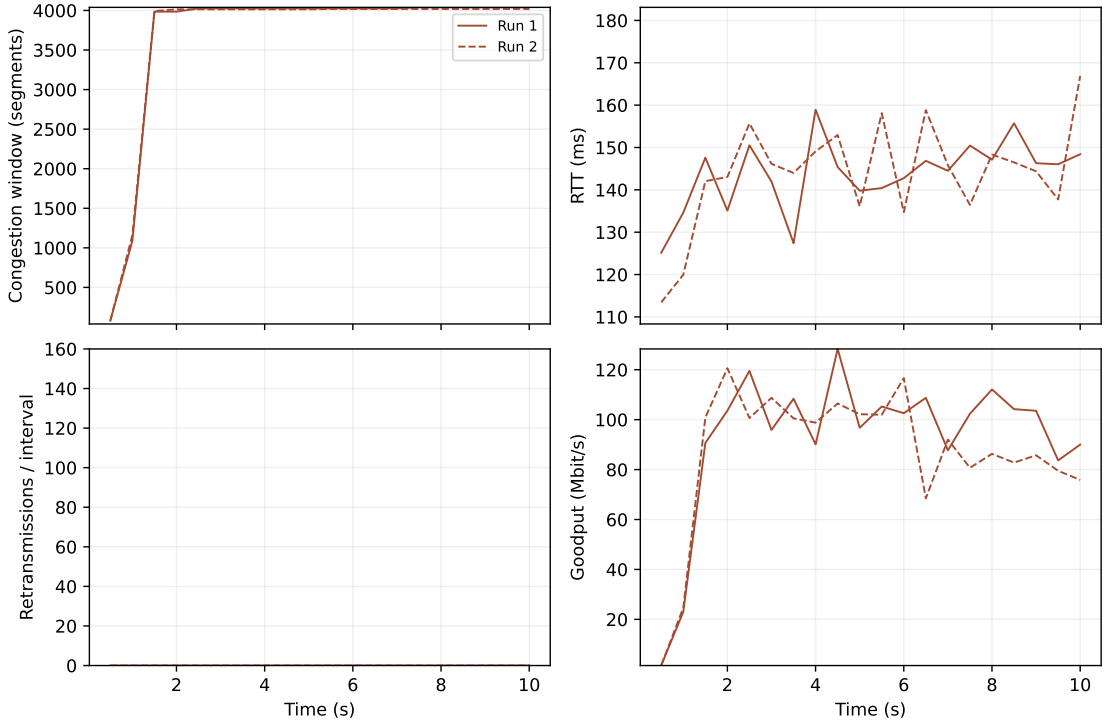


Figure 7: Reno congestion window, RTT, retransmissions per sample, and goodput over time at Toronto.

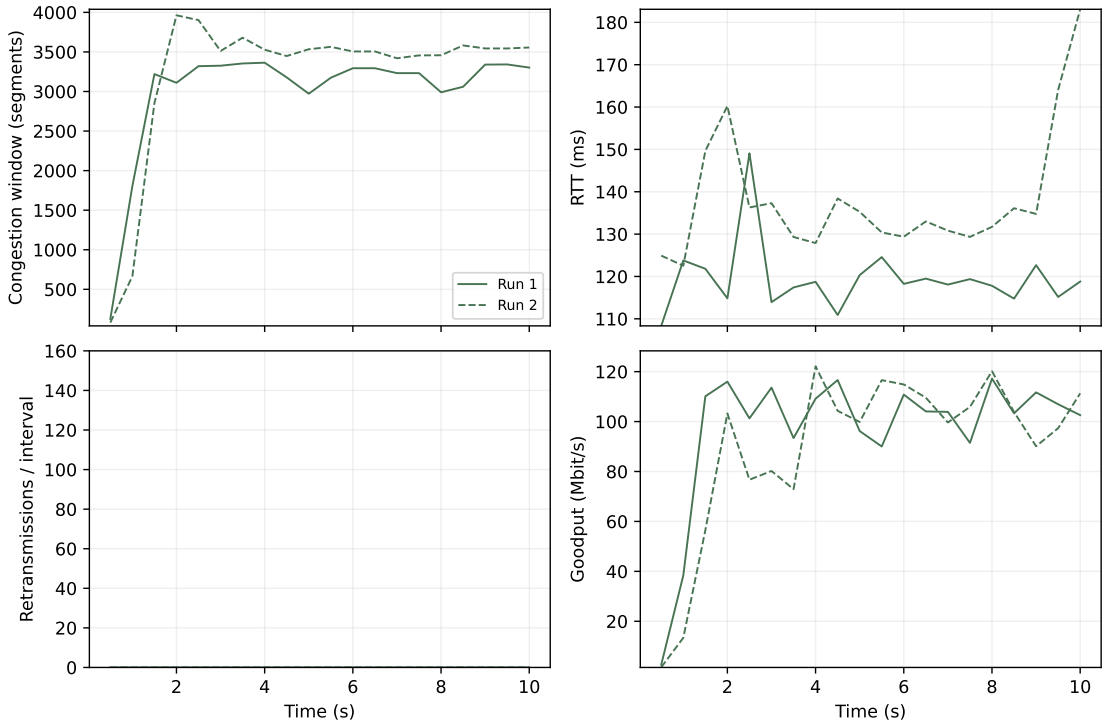


Figure 8: BBR congestion window, RTT, retransmissions per sample, and goodput over time at Toronto.

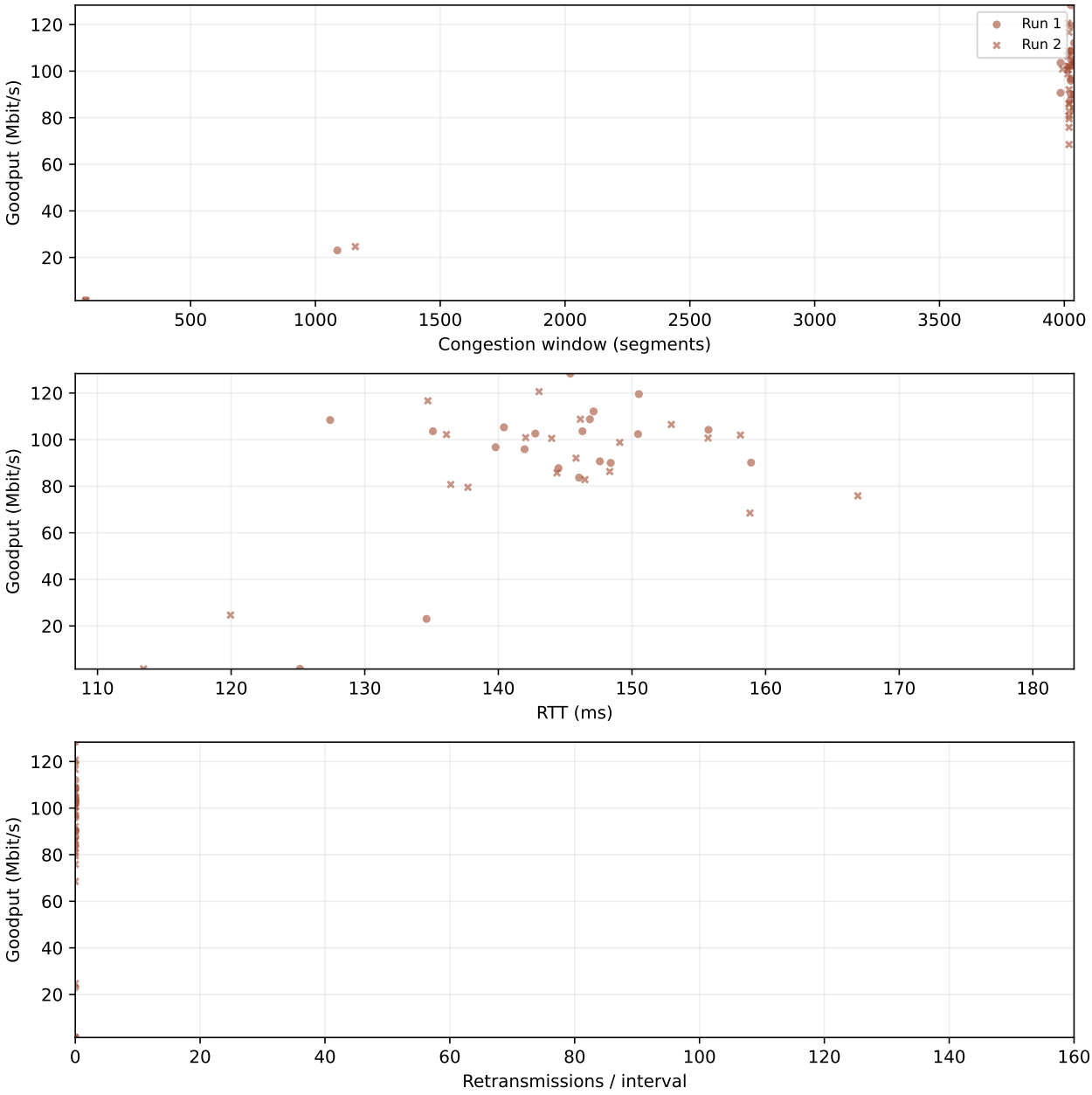


Figure 9: Reno congestion window, RTT, and retransmission signal versus acknowledged goodput at Toronto.

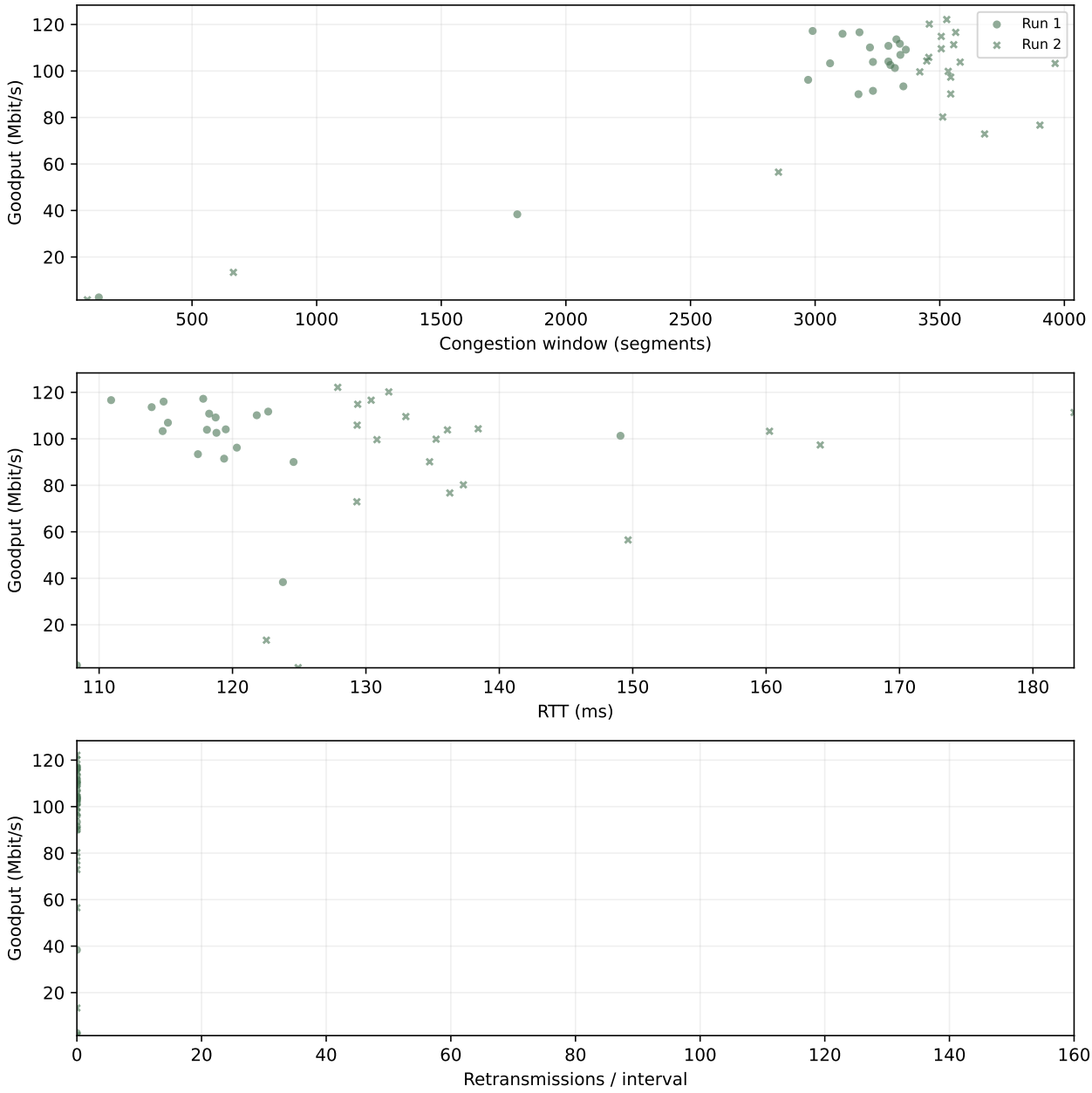


Figure 10: BBR congestion window, RTT, and retransmission signal versus acknowledged goodput at Toronto.

The same overshoot-and-hold pattern repeated on the medium-RTT paths. At Woodstock, kovacevic doubled to 640, cut once to 576, and held there with 102.7 and 104.7 Mbit/s median goodput, inside the 102 to 116 range of the other three. At the Miami server, one run exercised the full loss path. A retransmission halved the window from 640 to 320, the delay cut then trimmed it to 288, and the module held there with 100.8 Mbit/s, close to Reno's 110.8 and above BBR's 101.1 in that run. The Dallas server is the exception. Its first run went from 640 to 576 at 98.2 Mbit/s. The second run lost a packet early, halved to a window near 100 segments, and ended at 29.6 Mbit/s while CUBIC and Reno stayed above 103. kovacevic avoids loss on most paths, and when loss still happens, the halving plus the delay pause can leave it conservative for the rest of a short run.

At Vienna it reached 106.3 and 108.3 Mbit/s against CUBIC's 84.4 and 99.4. At Madrid it reached 113.2 and 109.7 against CUBIC's 64.3 and 26.6. At the Copenhagen server it reached 99.3 and 108.6, near BBR's 101.5 and 104.0, while CUBIC collapsed to 24.6 in one run. At the replacement destination, speedtest.xmission.com, the algorithms ran close together. kovacevic reached 107.6 and 107.7 against Reno's 112.9 and 108.2. The window in the traces is

a limit, not the measured amount in flight, so the queue size behind a held window is inferred from the RTT rise rather than read directly.

5. Limitations and reproducibility

The measurements have the same limits as the Assignment 2 runs. Public servers carry unknown load, the paths are uncontrolled, and matched tests run sequentially, so a goodput difference cannot be assigned to one mechanism alone. Ten-second transfers emphasize startup, which favors the fast starters. The module's thresholds, the 2 ms and 20% delay margin, the 5% flat threshold, and the 10% cut, were carried over from Assignment 2 and were not tuned against these paths. The delay pause does not remove an existing queue. It only stops growing, so a window that overshot the delay threshold once stays there, as the medium-RTT traces show. The module does not pace transmissions. Growth is ACK-clocked, so bursts can build short queues even while the window is held. A single flow cannot demonstrate fairness or convergence, which the Assignment 2 report discussed only as reasoning. The module was built and loaded only on this host's kernel, 6.17.0-35-generic.

The code is at github.com/kovaceviccz/cs422-assignment-3. The module registration carries the name and callbacks, and the main script runs the measurements and calls the plotting functions. Reproduction needs Linux with kernel headers matching the running kernel, the module loaded, CUBIC, Reno, and BBR available, Python 3.12, the packages in `requirements.txt`, and Typst 0.14.2. With those prerequisites and an Internet connection:

```
make && sudo insmod tcp_kovacevic.ko
git clone https://github.com/kovaceviccz/cs422-assignment-3
cd cs422-assignment-3
python3 -m pip install -r requirements.txt
python3 assignment3.py listed_iperf3_servers.json \
  --destinations 10 --duration 10 --interval 0.5 --repetitions 2
```

6. Conclusion

The Assignment 2 window rule runs in the Linux kernel as `kovacevic`. It doubles through startup, pauses on the RTT threshold, makes its one delay cut, and answers loss with a halving. Across ten destinations its median goodput across run medians matched Reno's and stayed above CUBIC's and BBR's. Its median RTT sat between BBR's and CUBIC's, and it recorded 3 retransmission intervals against Reno's 32 and CUBIC's 160, with BBR at 4. On the 187 ms Sydney path it held above 105 Mbit/s in both runs while CUBIC's first run stayed at 14.4. The weak point is the overshoot. On medium-RTT paths the doubling leaves a standing queue that the pause can only freeze, not drain, and a single 10% cut does not fully correct it.